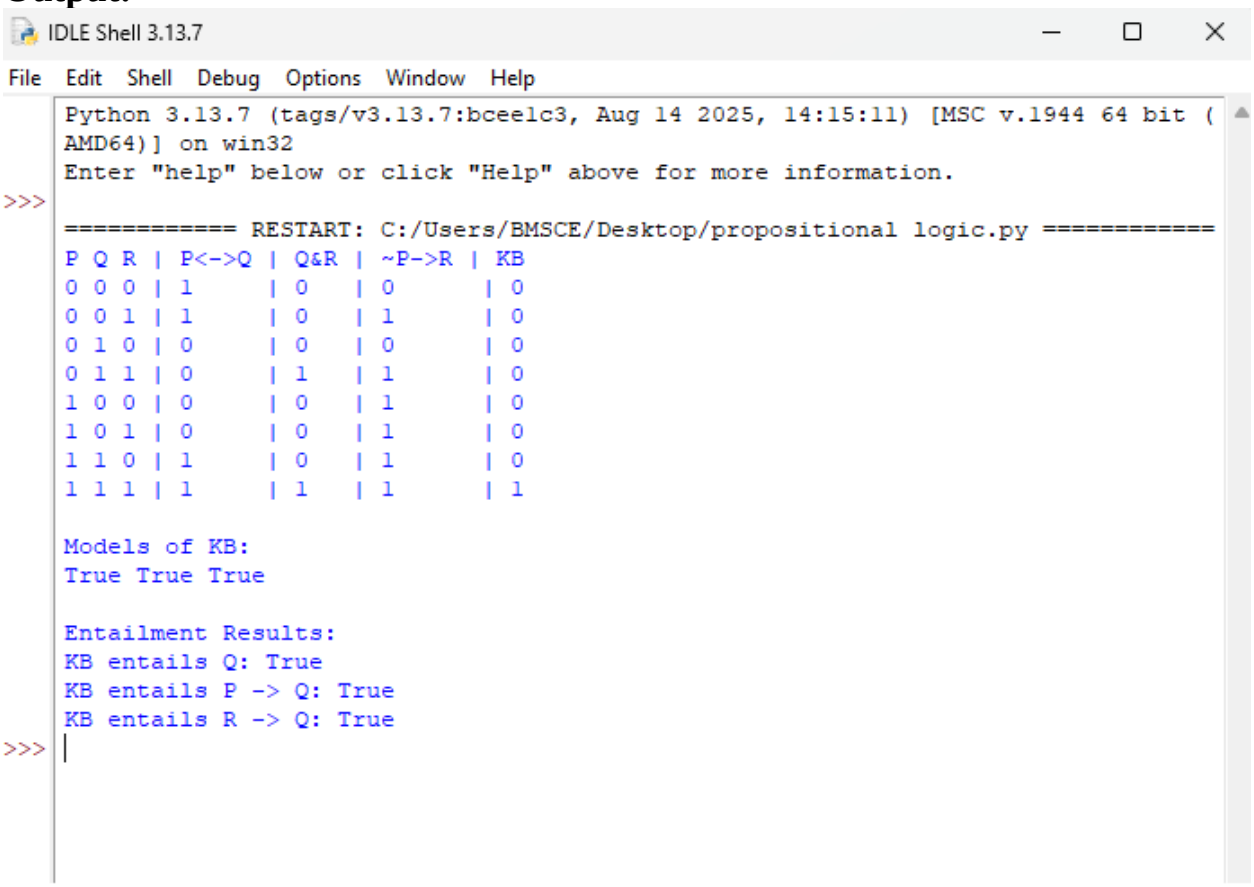
Propositional Logic

Code:

```
from itertools import product

def implies(a, b):
    return not a or b

def biconditional(a, b):
    return a == b

def KB(P, Q, R):
    return biconditional(P, Q) and (Q and R) and implies(not P, R)

models = list(product([False, True], repeat=3))

print("P Q R | P<->Q | Q&R | ~P->R | KB")

for P, Q, R in models:
    s1 = biconditional(P, Q)
    s2 = Q and R
    s3 = implies(not P, R)
    kb = KB(P, Q, R)

    print(int(P), int(Q), int(R), "|",
          int(s1), "    |",
          int(s2), "  |",
          int(s3), "     |",
          int(kb))

print("\nModels of KB:")

for P, Q, R in models:
    if KB(P, Q, R):
        print(P, Q, R)

def entails(query):
    for P, Q, R in models:
        if KB(P, Q, R) and not query(P, Q, R):
            return False
    return True

print("\nEntailment Results:")

print("KB entails Q:", entails(lambda P, Q, R: Q))
```

```
print("KB entails P -> Q:",
      entails(lambda P, Q, R: implies(P, Q)))

print("KB entails R -> Q:",
      entails(lambda P, Q, R: implies(R, Q)))
```

Output: